

Arithmetisches Schlussfolgern mit LLM: Prolog-Generierung & Permutation

Xiaocheng Yang und Bingsen Chen und Yik-Cheung Tam
Shanghai Frontiers Science Center of Artificial Intelligence and Deep Learning

New York University Shanghai{xy2128,bc3088,yt2267}@nyu.edu

Zusammenfassung

Die Anweisung großer Sprachmodelle (LLMs) zur Lösung von mathematischen Aufgaben der Grundschule hat großen Erfolg mit Chain of Thought (CoT) gezeigt. Der CoT-Ansatz basiert jedoch auf einem LLM, um eine Folge arithmetischer Berechnungen zu generieren, die zu kaskadierenden Berechnungsfehlern führen können. Wir nehmen an, dass sich ein LLM darauf konzentrieren sollte, Prädikate zu extrahieren und symbolische Formeln aus der mathematischen Aufgabenbeschreibung zu generieren, so dass die zugrunde liegende Berechnung über einen externen Code-Interpreter erfolgen kann. Wir untersuchen die Verwendung von LLM zur Generierung von Prolog-Programmen zur Lösung mathematischer Fragen. Experimentelle Ergebnisse zeigen, dass unsere Prolog-basierte arithmetische Problemlösung die CoT-Generierung im GSM8K-Benchmark übertrifft, und zwar über drei verschiedene LLMs hinweg. Darüber hinaus schlagen wir angesichts der unempfindlichen Reihenfolge von Prädikaten und symbolischen Formeln in Prolog vor, die Ground-Truth-Prädikate für ein robusteres LLM-Training durch Datenweiterung zu permutieren.

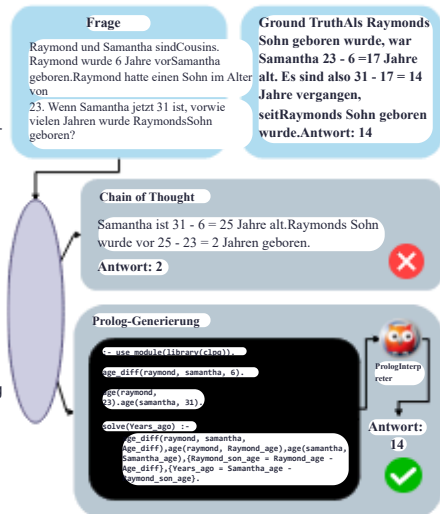


Abbildung 1: Überblick über die Prolog-Generierung für arithmetisches Schließen mit großen Sprachmodellen.

1 Einleitung

Große Sprachmodelle (LLMs) haben mit ihrer Skalierung der Modellgröße und Datengröße eine beeindruckende Leistung bei verschiedenen Verständnis- und Generierungsaufgaben gezeigt (Brown et al., 2020; Chowdhery et al., 2022; Rae et al., 2021; Thoppilan et al., 2022; Touvron et al., 2023; Almazrouei et al., 2023; Jiang et al., 2023). Dennoch versagen solche LLMs bei der Bearbeitung mathematischer Probleme, die arithmetische, alltagslogische und symbolische Schlussfolgerungen beinhalten – Themen, die dem Menschen täuschend einfach erscheinen mögen (Rae et al., 2021). Bestehende Arbeiten nutzen Chain-of-Thought (CoT)-Schlussfolgerungen, die Sprachmodelle auffordern, sowohl die Antwort als auch die schrittweise Argumentationskette zu generieren, was hilft, eine komplexe Argumentationsaufgabe in einen sequentiellen Denkprozess zu zerlegen (Wei et al., 2022b). Insbesondere hat sich gezeigt, dass arithmetisches Schließen mit CoT eine aufkommende Fähigkeit ist, die Sprachmodelle während des Skalierungsprozesses erworben haben (Wei et al., 2022a).

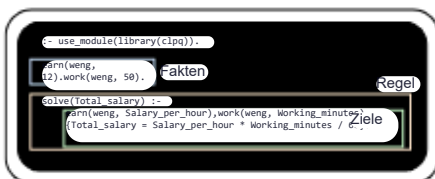
Allerdings ist die Verarbeitung natürlicher Sprache nicht nativ für mathematische Operationen und symbolische Manipulationen. Ein Forschungszweig hat sich darauf konzentriert, Sprachmodelle mit deterministischen Rechenressourcen wie einem Taschenrechner (Schick et al., 2023) oder programmgestützten Werkzeugen (Gao et al., 2023; Gou et al., 2023) zu erweitern. Alle diese Methoden erfordern jedoch einen sequenziellen Denkprozess, bei dem Modelle die natürlichsprachlichen Fragen in sequenzielle mathematische oder logische Operationen übersetzen müssen. Unsere Forschung untersucht die Anwendung von Prolog, einer logischen Programmiersprache, zur Lösung der Aufgabe des arithmetischen Denkens. Prolog löst arithmetische Denkaufgaben, indem es eine ungeordnete Menge von Prädikaten definiert und Abfragen darüber ausführt. Wir erläutern die einzigartigen Eigenschaften von Prolog in Abschnitt 2 weiter. Bei der Prolog-Code-Generierung für arithmetisches Denken extrahieren LLMs Fakten und Regeln in mathematischen Fragen -

tionen und sie in Prolog-Code umwandeln. Wenn die Fakten und Regeln korrekt erfasst werden, kann ein Prolog-Interpreter präzise eine korrekte Antwort deterministisch lösen.

Unsere Forschung hat die folgenden Beiträge: 1) Wir kuratieren und veröffentlichen das GSM8K-Prolog-Dataset mit einem halbautomatischen Ansatz, das arithmetische Denkaufgaben und die entsprechenden Prolog-Code-Lösungen enthält. 2) Unsere Experimente zeigen, dass die Prolog-Code-Generierung konsistent ist

besser als CoT bei der arithmetischen Denkaufgabe, was darauf hindeutet, dass sich LLM auf Prädikatsextraktionen konzentrieren und sich auf ein externes Tool verlassen kann, um zu berechnen und die logische Induktion durchzuführen, um mathematische Probleme zu lösen. 3) Angesichts der nicht-sequenziellen Natur von Prädikaten im Prolog-Code schlagen wir die Prädikatpermutation als eine Datenerweiterungsmethode vor und demonstrieren ihre Wirksamkeit im robusten LLM-Training.

Originale Grundwahrheit



Permutierte Grundwahrheit

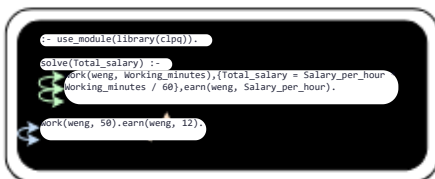


Abbildung 2: Prolog- und permutierte Prolog-Code-Beispiele.

2 Vorläufiges: Prolog-Sprache

Prolog ist eine logische Programmiersprache, die ursprünglich für künstliche Intelligenz und Computerlinguistik entwickelt wurde (Clocksin und Mellish, 2003; Bratko, 2012; Covington, 2002). Wie in der oberen Grafik von Abbildung 2 gezeigt, definiert ein Prolog-Programm eine Menge von Prädikaten, die Fakten und Ziele enthalten. In dem Beispiel beinhalten Fakten `earn(weng, 12)`, das das Stundengehalt von Weng deklariert, und `work(weng, 50)`, das die Arbeitsminuten von Weng definiert; die Ziele bilden eine Regel in der Form von `solve(answer):-<goal_1>,<goal_2>,...<A`

Eine Regel ist wahr, wenn alle Ziele erfüllt sind. Nachdem alle Fakten und Ziele im Programm definiert sind, können Benutzer eine Abfrage stellen, um die Lösungen zu erhalten, die die Regel angesichts aller Fakten wahr machen. Darüber hinaus sind Prolog-Codes nicht sequenziell wie Python, was bedeutet, dass die Reihenfolge der Fakten und Regeln das Ergebnis des Programms nicht verändert. Die untere Grafik in Abbildung 2 zeigt ein äquivalentes Beispiel, das die Reihenfolge der Prädikate permutiert, was das gleiche Ergebnis wie das Originalprogramm erzeugt.

3 Methode

3.1 GSM8K-Prolog-Dataset Nach unserem Wissen gibt es noch kein Dataset zum Lösen mathematischer Fragen mit Prolog. Wir haben daher ein Dataset basierend auf GSM8K (Cobbe et al., 2021), einem beliebten Benchmark für vielfältige mathematische Textaufgaben der Grundschule, auf halbautomatische Weise mit der Text Completion API 1 von OpenAI kuratiert. Insbesondere haben wir die gleichen Dataset-Splits und Fragen in GSM8K verwendet und GPT-4 aufgefördert, die Prolog-Programme zu generieren, um die Fragen zu lösen. Dann haben wir einige fehlerhafte Stichproben manuell korrigiert. Auf diese Weise haben wir einen hochwertigen Korpus mit 100% Genauigkeit in Bezug auf die Code-Ergebnisse erhalten. Algorithmus 1 beschreibt den detaillierten Pseudocode zur Erstellung dieses Datensets. Wir haben dieses Dataset der Forschungsgemeinschaft mit der MIT-Lizenz zur Verfügung gestellt. 2.

3.2 PROPER: Prolog-Permutation Da Prolog-Prädikate permutierbar sind, inspiriert von XLNet (Yang et al., 2020), das eine Token-weise Permutation über Aufmerksamkeitsmaskierung durchführt, haben wir beschlossen, auch die Permutationstechnik zu verwenden. Das XLNet kann über die Permutation während des Trainings auf Token auf beiden Seiten zugreifen und so teilweise die Eigenschaft der Autoenkodierung erhalten, während die Eigenschaft der autoregressiven Modellierung beibehalten wird. In ähnlicher Weise nutzt PROPER die permutative Eigenschaft von Fakten und Zielen in den Prolog-Programmen, wie in Abbildung 2 dargestellt. Für jedes Originalprogramm nehmen wir n seiner Permutationen als Stichprobe und mischen sie in das Dataset. Auf diese Weise können Modelle lernen, Prädikate in den mathematischen Fragen basierend auf allen anderen Prädikaten unabhängig von der Reihenfolge zu extrahieren, was die Natur genauer widerspiegelt

¹<https://platform.openai.com/docs/guides/text-generation/chat-completions-api>

²<https://huggingface.co/datasets/Thomas-X-Yang/gsm8k-prolog>

Methode	Llama-2		CodeLlama		Mistral	
	GSM8K	GSM-HARD	GSM8K	GSM-HARD	GSM8K	GSM-HARD
CoT	33.8%	12.0%	37.5%	13.9%	58.9%	30.8%
Prolog	41.5%	32.4%	55.0%	41.6%	66.3%	50.6%
KORREKT	51,0 %	37,4 %	59,0 %	45,9 %	70,2 %	54,4 %

Tabelle 1: Genauigkeitsergebnisse auf den GSM8K- und GSM-HARD-Datensätzen. Wir vergleichen die reguläre Prolog-Generierung (Prolog) und die PROPER Prolog-Generierung mit der CoT-Baseline (überwachtes Finetuning mit LoRA unter Verwendung von CoT-Ground-Truth-Labels im ursprünglichen GSM8K-Datensatz).

der Prolog-Sprache. Die praktischen Details der Permutation werden in Anhang A.2 beschrieben.

4 Experimente

4.1 AufbauDatensatzWir verwendeten das in Abschnitt 3.1 beschriebene GSM8K-Prolog. Wir bezeichnen den Korpus als D. Die Trainingsmenge ist Dtrain und die Testmenge ist Dtest. Die Gesamtgröße des Korpus beträgt 8792, wobei 7473 Stichproben zur Trainingsmenge und 1319 zur Testmenge gehören. Während des Trainings wurden 100 Stichproben aus der Trainingsmenge ausgewählt, um die Validierungsmenge zu bilden. Das Eingabeformat folgt dem in Stanford Alpaca (Taori et al., 2023) verwendeten Instruktionsprompt (siehe Beispielprompts in Anhang A.3). Wir haben Stichproben verworfen, die 512 Token überschritten. Insbesondere, als wir PROPER zur Erweiterung des Datensatzes verwendeten, verwendeten wir leicht veränderte Eingabeaufforderungen für permutierte Stichproben, da wir festgestellt haben, dass die Verwendung derselben Anweisung sowohl für die ursprünglichen Ground-Truth-Codes als auch für die permutierten die Leistung des Modells beeinträchtigte. Ein wahrscheinlicher Grund ist, dass mehrere korrekte Ausgabetoken für dieselbe Eingabeaufforderung das Modell verwirren. Darüber hinaus wurde neben dem Testset von GSM8K auch GSM-HARD (Gao et al., 2023) zur Auswertung verwendet, das die Zahlen im GSM8K-Testset durch große Zahlen ersetzt und somit Fragen für Sprachmodelle erschwert.

Training Wir experimentierten mit verschiedenen 7B-Versionen von LLMs, darunter Llama2 (Touvron et al., 2023), CodeLlama (Rozière et al., 2023) und Mistral (Jiang et al., 2023). Wir haben 8-Bit-Quantisierung und LoRA (Hu et al., 2021) verwendet, um Modelle effizient und zu einem angemessenen Leistungspreis zu finetunen. Wir haben LoRA angewendet, um Abfrage- und Wertgewichtsmatrizen in den Transformer-Blöcken zu finetunen. Wir experimentierten mit verschiedenen LoRA-Rängen und

Alpha-Einstellungen, einschließlich (r, α) = (8, 16), (16, 32) und (32, 64). Mit mehr trainierbaren Parametern ergab r = 32, α = 64 deutlich bessere Ergebnisse,

was wir daher als Konfiguration für alle Experimente übernommen haben. Beachten Sie, dass diese Einstellung dazu führte, dass nur 0,248 % der 7 Milliarden Parameter für Llama2 und CodeLlama und 0,188 % der 7 Milliarden für Mistral trainiert wurden. Wir dokumentieren unsere Trainingsdetails und die GPU-Nutzung in Anhang A.5.

Evaluierung Zur Inferenzzeit haben wir Beam verwendet Suche mit einer Strahlgröße von 4, um den Prolog-Code zu generieren. Wir verwendeten dann die PySwip-Bibliothek 3, eine Foreign-Interface von Prolog in Python, als Prolog-Interpreter, um die endgültige Antwort zu erzeugen. Wir verwendeten die Genauigkeit als Metrik für die Evaluierung. Sie ist definiert als

$$Acc = \frac{\sum_{i=1}^{|D_{test}|} 1_{n(P(a_{pre}^{(i)})=P(a_{true}^{(i)}))}}{|D_{test}|} \times 100 \%$$

wobei P den Prolog-Interpreter bezeichnet. Insbesondere, da wir festgestellt haben, dass die PySwip-Bibliothek keine Dezimalantworten verarbeiten kann, haben wir nur die Stichproben mit einer ganzzahligen Antwort berücksichtigt.

4.2 ErgebnisseDie Prolog-Generierung schneidet über alle drei Modelle hinweg durchweg besser ab als CoT. Gemäss Tabelle 1 führt die Generierung von Prolog zur Lösung mathematischer Fragen zu deutlich genaueren Ergebnissen, mit einer um 10,9 % höheren Genauigkeit gegenüber der CoT-Baseline im Durchschnitt aller Modelle auf GSM8K. Diese Lücke vergrössert sich auf 22,6 % auf GSM-HARD, was eine aussergewöhnliche Überlegenheit gegenüber CoT bei Berechnungen mit grossen Zahlen anzeigt. Obwohl Llama-2 und Mistral grosse Leistungslücken bei der Anwendung von CoT-Reasoning aufweisen, liefert die Generierung von Prolog-Code bessere Ergebnisse als CoT bei beiden Modellen. Diese Beobachtung deutet darauf hin, dass die Prolog-Generierung unabhängig von der inhärenten arithmetischen Denkfähigkeit des Modells gut funktioniert. Auch CodeLlama zeigt einen grösseren Leistungsgewinn beim Wechsel

³Prolog Version 9.0.4. PySwip Version 0.2.11. <https://github.com/yuce/pyswip>

Verhältnis =S	Llama-2	CodeLlama	Mistral
1:0	41.5	55.0	66.3
1:1	50.9 (49.5)	58.7 (56.6)	70.2 (69.1)
1:2	51.0 (49.4)	59.0 (58.3)	68.8 (66.8)

Abbildung 2: Ergebnisse zur Genauigkeit (%) auf GSM8K mit unterschiedlichen Permutationsverhältnissen. Wir berichten sowohl die beste als auch die durchschnittliche Genauigkeit von 1:1 und 1:2 über drei Versuche mit unterschiedlich zufällig permutierten Daten in der Form von max (avg). Beachten Sie, dass der Fall 1:0 im PROPER.

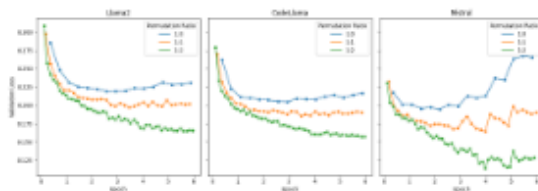


Abbildung 3: Validierungsverlustkurven für das Training von Llama2, CodeLlama und Mistral mit unterschiedlichen Permutationsverhältnissen (Wir berichten nur den ersten Versuch, wenn wir permutierte Daten verwenden, da die Verlustkurven über die Versuche hinweg sehr ähnlich sind).

von CoT zur Prolog-Generierung, was möglicherweise auf das Vortraining auf dem codebezogenen Korpus zurückzuführen ist. Mit anderen Worten, CodeLlama wurde speziell darauf trainiert, strukturierte Programme besser zu generieren als natürlichsprachliche Argumentationen.

Mit einem angemessenen Permutationsverhältnis verbessert PROPER die arithmetische Argumentation von LLM durch Prolog-Generierung weiter. Das Permutationsverhältnis bezieht sich auf das Verhältnis zwischen Originalmustern und permutierten Mustern. Wie in Tabelle 2 gezeigt, haben wir durch das Hinzufügen von zwei permutierten Mustern für jedes Originalmuster eine erhöhte Genauigkeit von 9,5 % bzw. 4,0 % für Llama-2 und CodeLlama im Testdatensatz beobachtet. Diese Verbesserung deutet darauf hin, dass das Erlernen der nicht-sequenziellen Struktur von Prolog-Prädikaten für LLMs hilfreich ist, um korrekte Prolog-Programme zur Lösung arithmetischer Probleme zu generieren. Andererseits deutet die verringerte Genauigkeit von Mistral im Vergleich zu seinem Fall einer Permutation pro Muster darauf hin, dass PROPER für Modelle mit bereits hoher Prolog-Generierungskapazität möglicherweise begrenzt ist.

Ein geringerer Validierungsverlust durch PROPER führt nicht zu einer höheren Genauigkeit. Wie in Abbildung 3 gezeigt, führt die Erhöhung des Permutationsverhältnisses zu einem deutlich geringeren Validierungsverlust. Dies liegt daran, dass wir zuerst Permutationen hinzugefügt und dann einen Validierungsdatensatz aus dem Trainingsdatensatz aufgeteilt haben. Folglich wurden die Permutationen der Validierungsmuster in den Trainingsdatensatz aufgenommen, und die Generalisierungsfähigkeit der Sprachmodelle ermöglichte es den Modellen, die Permutationen zu nutzen, um die Leistung im Validierungsdatensatz zu verbessern, was zu einem leichten Datenleck führte. Daher ergab das Permutationsverhältnis von 1:2 gemäß Tabelle 2 eine schwächere Leistung bei Mistral, obwohl der Validierungsverlust am geringsten war.

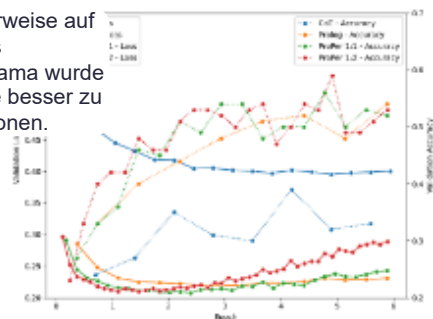


Abbildung 4: Validierungsverlustkurven und Validierungsgenauigkeitskurven für das Training von Llama2 mit verschiedenen Methoden (Wir berichten nur den ersten Versuch, wenn wir permutierte Daten verwenden, da die Verlust- und Genauigkeitskurven über die Versuche hinweg sehr ähnlich sind).

Ein erhöhter Validierungsverlust durch PROPER führt nicht zu einer verringerten Validierungsgenauigkeit. Unter Ausschluss der Permutationen von Validierungsmustern aus dem Trainingsdatensatz berichten wir sowohl den Kreuzentropieverlust als auch die Genauigkeit im Validierungsdatensatz für Llama2 unter Verwendung verschiedener Methoden in Abbildung 4. Es wird eine Diskrepanz zwischen Verlust und Genauigkeit beobachtet. Wenn eine Verlustkurve auf das Minimum sinkt und zurückprallt, steigt die entsprechende Genauigkeitskurve weiter an und hält dann ein hohes Niveau. Wie in Tabelle 3 gezeigt, kann die Leistung durch die Auswahl von Checkpoints basierend auf der Validierungsgenauigkeit anstelle des Validierungsverlusts über alle Methoden hinweg verbessert werden. Darüber hinaus ist die Verbesserung für die Prolog- und PROPER-Methode deutlich größer als die von CoT, was auf eine größere Divergenz zwischen dem Ziel des Kreuzentropieverlusts und der letztendlichen Genauigkeit der Prolog-Generierung hindeutet. Daher wird vorgeschlagen, den besten Checkpoint basierend auf dem zu wählen

Methode	Initial	Keine Leckage (durch Verlust) (durch Genauigkeit)	Keine Leckage (durch Verlust) (durch Genauigkeit)
CoT	33.8	33.8	36.5
Prolo	41.5	41.5	47.9
ProPer 1:1	50.9 (49.5)	44.3 (43.4)	50.1 (48.4)
ProPer 1:2	51.0 (49.4)	44.4 (43.6)	51.3 (50.3)

Tabelle 3: Ergebnisse der Genauigkeit (%) des Trainings von Llama2 auf dem GSM8K-Datensatz. Wir vergleichen die Ergebnisse der Vermeidung von Validierungs-Sample-Leckagen im Trainingsdatensatz und der Auswahl des optimalen Checkpoints basierend auf Validierungsverlust und -genauigkeit mit den anfänglichen Ergebnissen mit Leckagen. Die beste und durchschnittliche Genauigkeit von 1:1 und 1:2 sind in der Form von max (durchschnittlich).

Validierungsgenauigkeit. Dennoch ist die neue Leistung ähnlich den anfänglichen Ergebnissen, bei denen Leckagen vorliegen. Wir stellen fest, dass späte Checkpoints eine bessere Leistung gemäß der Validierungsgenauigkeit erzielen und der Validierungsverlust in der anfänglichen Einstellung weiter abnimmt. Daher wählen beide Einstellungen zufällig späte Checkpoints aus, was zu einer ähnlichen Leistung führt.

Wir haben auch die Python-Generierung getestet, für die der Korpus nach dem gleichen Verfahren wie Algorithmus 1 generiert wurde, außer dass wir Python-Codes anstelle von Prolog-Codes vorbereiten. Dies ergibt eine Genauigkeit von 55,12 % auf GSM8K unter Verwendung von Llama2 als Basismodell, was besser ist als sowohl Prolog als auch PROPER. Ein möglicher Grund ist, dass Python jetzt die vorherrschende Programmiersprache ist und Llama2 möglicherweise auf einer großen Menge von Python-Codes vortrainiert wurde. Wir glauben, dass, wenn ausreichend Prolog-Codes für

Das Training verwendet wird, kann die Prolog-Generierung aufgrund ihres Wesens der symbolischen Argumentation zumindest mit der Python-Generierung mithalten.

Wir präsentieren einige repräsentative Fehlerfälle von Mistral (1:1) in Anhang A.4.

5 Verwandte Arbeiten

Arithmetisches Schließen Der Chain-of-Thought(CoT)-Prompting-Ansatz (Wei et al., 2022b) schlägt zunächst vor, das Modell aufzufordern, die Schlussfolgerungskette Schritt für Schritt zu generieren, um die endgültige Antwort zu erhalten. Danach wurden Fortschritte in der Schlussfolgerungsfähigkeit von LLMs durch schrittweise Methoden erzielt (Zhou et al., 2023; Zhu et al., 2023; Huang et al., 2022; Liang et al., 2023). Die Generierung natürlicher Sprache schneidet jedoch bei komplexen oder mehrstufigen Schlussfolgerungen immer noch schlecht ab. Daher wurden Anstrengungen unternommen, um Schlussfolgerungsstrukturen wie Bäume (Yao et al., 2023; Long,

2023) und Graphen (Besta et al., 2023; Zhang et al., 2023) zu nutzen. Ein weiterer Ansatz ist die Gestaltung der Schlussfolgerungsaufgabe auf der Grundlage externer Tools (Cobbe et al., 2021; Mishra et al., 2023; Gou et al., 2023; Gao et al., 2023; Shao et al., 2023; Chen et al., 2023), was derjenige ist, dem wir folgen. Darüber hinaus teilt Yuan et al.s (2023) RFT-Methode die Idee der Datenerweiterung, aber sie kompilieren Ablehnungstichproben aus mehreren Modellen, um einen erweiterten Trainingsdatensatz zu bilden, der sich von PROPERs automatischer Permutation unterscheidet.

Neurales symbolisches Schließen Das neurale symbolische Schließen (Andreas et al., 2016; Neelakantan et al., 2017; Hudson und Manning, 2019; Gupta et al., 2020; Nye et al., 2021) zielt darauf ab, sowohl neuronale Netze als auch symbolisches Schließen zu nutzen, um bessere Schlussfolgerungsfähigkeiten und Transparenz zu erzielen. Diese Methoden leiden unter der geringen Skalierbarkeit von Lern- und Schlussfolgerungskomponenten. LLMs werden daher eingesetzt, um symbolische Darstellungen aus natürlicher Sprache zu generieren (Lyu et al., 2023; Pan et al., 2023; Yang et al., 2023), wobei deterministische symbolische Löser die Abfrage und die von LLMs generierten symbolischen Darstellungen verarbeiten, um Schlussfolgerungen oder Beweise durchzuführen. Prolog ist ein beliebter Kandidat für das Format symbolischer Darstellungen. Wir bauen auf diesem Ansatz und im speziellen Bereich des arithmetischen Schließens auf.

6 Fazit

Zusammenfassend lässt sich sagen, dass wir die Schlussfolgerungsleistung von LLMs verbessern wollen. Wir verwenden die Pipeline, bei der das Modell Prolog-Prädikate aus einer mathematischen Frage in natürlicher Sprache generiert und ein externer Prolog-Interpreter die Abfrage für ein Endergebnis verarbeitet. Wir stellen einen Open-Source-Korpus namens GSM8K-Prolog zur Verfügung, eine hochwertige, mit Prolog annotierte Version von GSM8K. Wir zeigen, dass die Prolog-Generierung die CoT-Generierung bei allen drei 7B-Modellen zur Lösung arithmetischer Schlussfolgerungsprobleme deutlich übertraf. Wir schlagen auch PROPER vor, eine Datenerweiterungsmethode, die speziell für die Prolog-Code-Generierung entwickelt wurde und es den feinabgestimmten Modellen ermöglicht, die nicht-sequentielle Natur von Prolog-Prädikaten zu erlernen. PROPER verbessert die Genauigkeit des Modells bei GSM8K-Prolog weiter und mildert die frühe Konvergenz während des Trainings. Schließlich schlagen wir aufgrund der Diskrepanz zwischen der Cross-Entropy-Loss-Zielfunktion und der Genauigkeit vor, die Validierungsgenauigkeit anstelle des Validierungsverlusts zu verwenden, um den besten Checkpoint auszuwählen.

Einschränkungen

Obwohl wir experimentell Full-Parameter-Finetuning durchgeführt haben, war das Ergebnis nicht zufriedenstellend. Wir glauben, dass dies an der begrenzten Größe des

ursprünglichen Korpus liegt. Daher können wir zum jetzigen Zeitpunkt keinen Vergleich mit anderen Methoden wie ToRA (Gou et al., 2023) oder RFT (Yuan et al., 2023) durchführen. Zukünftige Forschung kann sich mit der Erstellung eines größeren und vielfältigeren Korpus befassen, der an die Prolog-Code-Generierung angepasst ist. Außerdem haben wir nicht versucht, das Basismodell auf mehr als 7B Parameter zu skalieren. Daher kennen wir die Auswirkungen der Modellskalierung auf die Leistung der Prolog-Code-Generierung für arithmetische Schlussfolgerungen nicht. Aufgrund der Einschränkung der PySwinLibrary sind lösbare Fragen auf solche mit einer ganzzahligen Antwort beschränkt. Zukünftige Arbeiten können den Bereich durch die Verwendung anderer Interpretationswerkzeuge erweitern.

Referenzen

- Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Merouane Debbah, Etienne Goffinet, Daniel Hestlow, Julien Launay, Quentin Lalaric, Badreddine Noune, Baptiste Pannier und Guilherme Penedo. 2023. Falcon-40B: ein offenes, großes Sprachmodell mit erstklassiger Leistung.
- Jacob Andreas, Marcus Rohrbach, Trevor Darrell und Dan Klein. 2016. Neuronale Modulnetzwerke. In 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Seiten 39–48.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr Nyczyk und Torsten Hoefler. 2023. Graph of Thoughts: Solving elaborate problems with large language models.
- Ivan Bratko. 2012. Prolog-Programmierung für künstliche Intelligenz. Addison-Wesley.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Sprachmodelle sind Few-Shot-Lerner. *Advances in neural information processing systems*, 33:1877–1901.
- Wenhu Chen, Xueguang Ma, Xinyi Wang und William W. Cohen. 2023. Programm der Gedanken Prompting: Entwirren der Berechnung von der Argumentation für numerische Argumentationsaufgaben.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov und Noah Fiedel. 2022. Palm: Skalierung der Sprachmodellierung mit Pfaden.
- W. F. Clocksin und C. S. Mellish. 2003. Programmierung in Prolog. Springer-Verlag.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse und John Schulman. 2021. Training von Verifizierern zur Lösung von mathematischen Textaufgaben.
- Michael A. Covington. 2002. Verarbeitung natürlicher Sprache für Prolog-Programmierer. Prentice Hall.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan und Graham Neubig. 2023. Pal: Programmgestützte Sprachmodelle.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujia Yang, Minlie Huang, Nan Duan und Weizhu Chen. 2023. Tora: Ein toolintegrierter Denkkagent zur Lösung mathematischer Probleme.
- Nitish Gupta, Kevin Lin, Dan Roth, Sameer Singh, und Matt Gardner. 2020. Neuronale Modulnetzwerke für das Schließen über Text. In *International Conference on Learning Representations*.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang und Weizhu Chen. 2021. Lora: Low-Rank-Adaption von großen Sprachmodellen.
- Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu und Jiawei Han. 2022. Große Sprachmodelle können sich selbst verbessern.
- Drew Hudson und Christopher D Manning. 2019. Lernen durch Abstraktion: Die neuronale Zustandsmaschine. In *Advances in Neural Information Processing Systems*, Band 32. Curran Associates, Inc.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao,

- Thibaut Lavril, Thomas Wang, Timothée Lacroix und William El Sayed. 2023. Mistral 7b. Machine Learning, Band 202 der Proceedings of Machine Learning Research, Seiten 30706–30775. PMLR.
- Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu und Shuming Shi. 2023. Förderung divergenten Denkens in großen Sprachmodellen durch Multi-Agenten-Debatte.
- Jieyi Long. 2023. Von großen Sprachmodellen geleiteter Baum-des Denkens.
- Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki und Chris Callison-Burch. 2023. Getreue Chain-of-Thought-Argumentation.
- Swaroop Mishra, Matthew Finlayson, Pan Lu, Leonard Tang, Sean Welleck, Chitta Baral, Tanmay Rajpurohit, Oyvind Tafford, Ashish Sabharwal, Peter Clark und Ashwin Kalyan. 2023. Lila: Ein vereinheitlichter Benchmark für mathematisches Denken.
- Arvind Neelakantan, Quoc V. Le, Martin Abadi, Andrew McCallum und Dario Amodei. 2017. Erlernen einer natürlichen Sprachschnittstelle mit neuronalem Programmierer. In International Conference on Learning Representations.
- Maxwell Nye, Michael Henry Tessler, Joshua B. Tenenbaum und Brenden M. Lake. 2021. Verbesserung der Kohärenz und Konsistenz in neuronalen Sequenzmodellen mit Dual-System, neuro-symbolischem Denken.
- Liangming Pan, Alon Albalak, Xinyi Wang und William Yang Wang. 2023. Logic-lm: Stärkung großer Sprachmodelle mit symbolischen Lösern für getreues logisches Denken.
- Jack W Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. 2021. Skalierung von Sprachmodellen: Methoden, Analyse & Erkenntnisse aus dem Training von Gopher. arXiv Preprint arXiv:2112.11446.
- Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wen-han Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom und Gabriel Synnaeve. 2023. Codellama: Offene Basismodelle für Code.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda und Thomas Scialom. 2023. Toolformer: Sprachmodelle können sich selbst beibringen, Werkzeuge zu benutzen. arXiv Preprint arXiv:2302.04761.
- Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan und Weizhu Chen. 2023. Synthetisches Prompting: Generieren von Chain-of-Thought-Demonstrationen für große Sprachmodelle. In Proceedings of the 40th International Conference on
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang und Tatsunori B. Hashimoto. 2023. Stanford Alpaca: Ein anweisungsfolgendes Llama-Modell. https://github.com/tatsu-lab/stanford_alpaca.
- Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. 2022. Lamda: Sprachmodelle für Dialoganwendungen. arXiv Preprint arXiv:2201.08239.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shrusti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurolien Rodriguez, Robert Stojnic, Sergey Edunov und Thomas Scialom. 2023. Llama 2: Offene Foundation- und feinabgestimmte Chatmodelle.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean und William Fedus. 2022a. Emergent Fähigkeiten großer Sprachmodelle.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022b. Chain-of-Thought-Prompting löst Schlussfolgerungen in großen Sprachmodellen aus. Advances in Neural Information Processing Systems, 35:24824–24837.
- Sen Yang, Xin Li, Leyang Cui, Lidong Bing und Wai Lam. 2023. Neuro-symbolische Integration bringt kausale und zuverlässige Beweise für Schlussfolgerungen.
- Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov und Quoc V. Le. 2020. Xlnet: Verallgemeinertes autoregressives Pretraining für das Sprachverständnis.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao und Karthik Narasimhan. 2023. Tree of Thoughts: Überlegtes Problemlösen mit großen Sprachmodellen.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Chuanqi Tan und Chang Zhou. 2023. Skalierungsbeziehung beim Erlernen mathematischen Denkens mit großen Sprachmodellen. arXiv-Vorabdruck arXiv:2308.01825.

Yifan Zhang, Jingjin Yang, Yang Yuan und Andrew Chi-Chih Yao. 2023. Kumulatives Denken mit großen Sprachmodellen.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le und Ed Chi. 2023. Least-to-most Prompting ermöglicht komplexes Denken in großen Sprachmodellen.

Xinyu Zhu, Junjie Wang, Lin Zhang, Yuxiang Zhang, Yongfeng Huang, Ruyi Gan, Jiaxing Zhang und Yujia Yang. 2023. Lösen von mathematischen Textaufgaben durch kooperatives, induziertes Sprachmodell. In Proceedings des 61. Jahrestreffens der Association for Computational Linguistics (Band 1: Lange Artikel). Association for Computational Linguistics.

Ein Anhang

A.1 Generierungsverfahren des GSM8K-PrologNachfolgend finden Sie den detaillierten Pseudo-Code für die Generierung des GSM8K-Prolog-Datensatzes.

Algorithmus 1 Ablauf der GSM8K-Prolog-GenerierungEingabe: Der

ursprüngliche GSM8K-Datensatz, bezeichnet als Satz $X = \{(q_i, aCoTi)\}_{i=1}^N$, wobei jede Probe aus
aus einer Frage q_i und einer Gedankenketten-Antwort $aCoTi$; Einen Prolog-Interpreter P, der die
Ausgabe eines Prolog-Programms; Ein Chain-of-Thought-Antwort-Retriever C, der die
endgültige Antwort einer Denkkette in natürlicher Sprache analysiert. Ausgabe: GSM8K-
Prolog-Datensatz $D = \{(q_i, aPrologi)\}_{i=1}^N$

Initialisieren Sie einen Satz von Indizes $I \leftarrow \{1, \dots, N\}$, eine statische
Eingabeaufforderung in den neuen Datensatz-Pins und eine erste Frage zum Abfragen der
OpenAI-API q_{gen} . Erstellen Sie manuell 10 korrekte Prolog-Codes $\{aPrologi\}_{i=1}^{10}$, die
 $\{q_i\}_{i=1}^{10}$ in X korrekt lösen, um D für i zu initialisieren, $i \in \{1, \dots, 10\}$

```
entnehmen Sie eine Probe  $(q_i, aCoTi) \in X$ 
GPT-4 mit  $\{q_{gen}\} \cup \{(q_k, aCoTk, aPrologk) \mid k=1, \dots, 10\} \cup \{q_i\}$  um ein Prologi zu erhalten
wenn  $P(aPrologi) = C(aCoTi)$  und dann
 $D \leftarrow D \cup \{(Pins, q_i, aPrologi)\}$ 
```

$I \leftarrow I \setminus \{i\}$ end ifend forManuell die Top 10 sauberen und logischen Prolog-Codes aus dem
aktuellen D auswählen, um ein neues Few-Shotsample-Set zu bilden $Q_{fixed} = \{(q_k, aCoTk, aPrologk) \mid k \in I\}$, $|Q_{fixed}| = 10$. für $j = 1, \dots, M$ do// M Versuchsversuche

```
denn ich  $i \in I$  tue
entnehmen Sie eine Probe  $(q_i, aCoTi) \in X$ 
Stichprobe Random  $\leftarrow \{(q_k, aCoTk, aPrologk) \mid k \in I\}$ ,  $|Q_{random}| = 10$  von D
// Hinzufügen von 10 dynamischen und 10 festen Beispielen zum 20-
Shot-Prompt. Prompt GPT-4 mit  $\{q_{gen}\} \cup Q_{fixed} \cup Q_{random} \cup \{q_i, aCoTi\}$ . um  $aPrologi$  if  $P(aPrologi) = C(aCoTi)$  zu erhalten, dann  $D \leftarrow D \cup \{(Pins, q_i, aPrologi)\}$ 
```

$I \leftarrow I \setminus \{i\}$ end ifend
forend forif $V = \emptyset$ then

```
Manuelle Korrektur der Prolog-Codes  $\{aPrologi\}_{i \in I}$ , die  $\{q_i\}_{i \in I}$  lösen
 $D \leftarrow D \cup \{(Pins, q_i, aPrologi) \mid i \in I\}$ 
```

end if

A.2 PermutationsverfahrenPermutationen können sowohl auf der Ebene von Fakten oder Regeln als auch auf der Ebene von Zielen in einer Regel durchgeführt werden. In der Praxis permutieren wir für jedes Codefragment zuerst die Ziele im solve<answer>:-<goal_1>,<goal_2>,... Prädikat. Da die Gesamtzahl der Permutationen von der Anzahl der Ziele abhängt und leicht auf eine große Größenordnung anwachsen kann, was zu einem Speicherüberlauf führen kann, haben wir die Permutationsmethode in der itertoolslibrary verwendet, um einen Iterator über die Permutationen zu erzeugen. Dann haben wir bis zu 10 Zielpermutationen aus dem Iterator entnommen. Wenn es insgesamt weniger als 10 Zielpermutationen gab, weil der Code prägnant war und es nicht viele Ziele gab, haben wir so viele Zielpermutationen wie möglich entnommen. Dann haben wir auf die gleiche Weise bis

bis zu 10 Fakten- und Regelpermutationen. Im Prinzip würden maximal 100 permutierte Stichproben für eine ursprüngliche Stichprobe generiert. Dann haben wir für jede Stichprobe, während wir ein Experiment durchführten, das eine bestimmte Anzahl von Permutationen erforderte, zufällig Permutationen aus der Menge der Permutationen bis zu einer Größe von 100 entnommen. Wenn für eine Stichprobe die Zielanzahl der Permutationen die Gesamtzahl der Permutationen überstieg, haben wir stattdessen alle ihre Permutationen verwendet.

A.3 Beispiele für Eingabeaufforderungen

Im Folgenden sind die Anweisungsaufforderungen aufgeführt, die wir für verschiedene Trainingsumgebungen verwendet haben (CoT, Prolog und PermutedProlog).

Einstellung	Prompt-Vorlage
CoT	Nachfolgend finden Sie eine Anweisung, die eine Aufgabe beschreibt, zusammen mit einer Eingabe, die weitere Informationen liefert. Schreiben Sie eine Antwort, die die Anfrage angemessen ausfüllt.### Anweisung:Bitte generieren Sie eine erklärende Antwort zur Lösung des gegebenen mathematischen Problems.### Eingabe:<Frage> ### Ausgabe:<CoT-Argumentation>Prolog-GenerierungNachfolgend finden Sie eine Anweisung, die eine Aufgabe beschreibt, zusammen mit einer Eingabe, die weitere Kontext liefert. Schreiben Sie eine Antwort, die die Anfrage angemessen ausfüllt.### Anweisung:Bitte generieren Sie ein Stück Prolog-Code, um das gegebene mathematische Problem zu lösen.### Eingabe:<Frage> ### Ausgabe:<Prolog-Code>
Permutierte Prolog-Generierung	Nachfolgend finden Sie eine Anweisung, die eine Aufgabe beschreibt, zusammen mit einer Eingabe, die weitere Informationen liefert. Schreiben Sie eine Antwort, die die Anfrage angemessen ausfüllt.### Anweisung:Bitte generieren Sie ein Stück Prolog-Code in nicht-sequenzieller Reihenfolge, um das gegebene mathematische Problem zu lösen.### Eingabe:<Frage> ### Ausgabe:<Prolog-Code>

A.4 Fehleranalyse

In diesem Abschnitt stellen wir einige typische Fehler vor, die von unserem leistungsstärksten Modell gemacht wurden, um das Verhalten und den Engpass der LLM-Generierung von Prolog-Programmen zur Lösung arithmetischer Denkaufgaben zu verstehen. Wir haben 70 problematische Prolog-Codes untersucht, die von finetuned Mistral unter Verwendung der 1:1-Permutationsumgebung generiert wurden, die die höchste Genauigkeit erzielt hatte, Fehler aufwies und manuell die Art der Fehler mithilfe des Swi-Prolog-Interpreters überprüft. 100 % von ihnen wiesen semantische Fehler auf, was bedeutet, dass ihre Lösungslogik falsch war. 7 % von ihnen wiesen Syntaxfehler auf, was bedeutet, dass Swi-Prolog bei diesen Codes Syntaxfehler ausgibt. Einige Beispiele für typische Syntaxfehler sind unten aufgeführt. Die Zeilen, die Fehler verursachen, sind fett gedruckt. Die Details der Fehler werden in den Kommentaren erläutert.

Frage	Prolog-Code
Julias Boot schlug leck, als sie auf dem See war. Das Boot nahm zwei Liter Wasser für jeweils zehn Fuß auf, die sie zurück zum Ufer ruderte. Sie brauchte sechzehn Sekunden, um zwanzig Fuß zu rudern. Das Ufer war 64 Sekunden entfernt. Wie viel Wasser hatte das Boot aufgenommen, als Julia das Ufer erreichte?	<pre> :- use_module(library(clpqr)). liters_per_10_feet(julia, 2). time_per_20_feet(julia, 16). distance_to_shore(julia, 64). solve(Water_taken_on) :-liters_per_10_feet(julia, Liters_per_10_feet),time_per_20_feet(julia, Time_per_20_feet),distance_to_shore(julia, Distance_to_shore),{Seconds_per_10_feet = Time_per_20_feet / 2},{Total_seconds = Distance_to_shore / 10 * Seconds_per_10_feet}, {Total_feet_rowed = Total_seconds / Seconds_per_20_feet * 20}, % Die Variable wurde noch nie definiert.(Total_liters_taken_on = Total_feet_rowed * Liters_per_10_feet),(Water_taken_on = Total_liters_taken_on). </pre>
Bill backt jeden Tag 300 Brötchen, 120 Schokoladen-Croissants und 60 Baguettes. Jedes Brötchen ist 4 Zoll lang, jedes Croissant ist 6 Zoll lang und jedes Baguette ist zwei Fuß lang. Wenn Bill alle Backwaren aneinander legt, wie lang werden sie dann in Fuß sein?	<pre> :- use_module(library(clpqr)). bake(bill, rolls, 300),bake(bill, chocolate_croissants, 120),bake(bill, baguettes, 60). length(roll, 4). # Das Prädikat ist bereits von Prolog reserviert.length(chocolate_croissant, 6),length(baguette, 2). solve(Total_length) :-bake(bill, rolls, Num_rolls),bake(bill, chocolate_croissants, Num_chocolate_croissants),bake(bill, baguettes, Num_baguettes),length(roll, Roll_length),length(chocolate_croissant, Chocolate_croissant_length),length(baguette, Baguette_length),(Roll_total_length = Num_rolls * Roll_length), {Chocolate_croissant_total_length = Num_chocolate_croissants * Chocolate_croissant_length},{Baguette_total_length = Num_baguettes * Baguette_length},{Total_length = Roll_total_length + Chocolate_croissant_total_length + Baguette_total_length}.% Dieser Code enthält auch einen semantischen Fehler, da er die Einheit Zoll nicht in Fuß umrechnet. </pre>
In einem Rennen mit 240 Asiaten waren 80 Japaner und der Rest Chinesen. Wenn die Anzahl der Jungen im chinesischen Team 60 betrug, wie viele Mädchen waren dann im chinesischen Team?	<pre> :- use_module(library(clpqr)). total_participants(240),japanese_p articipants(80),chinese_boys(60). solve(Girls_on_chinese_team) :- total_participants(Total_participants),Japanese_participants , % Die korrekte Syntax: japanese_participants(Japanese_participants},{Chinese_participants = Total_participants - Japanese_participants},chinese_boys(chinese_boys), {Girls_on_chinese_team = Chinese_participants - Chinese_boys}. </pre>
Brinley ist in Mr. Berts Matheunterricht. Mr. Bert gibt jedes Semester sechs Tests. Er erlaubt den Schülern, die niedrigste Punktzahl aus dem Durchschnitt jedes Semesters zu entfernen. Wenn Brinley in den ersten fünf Tests die Punktzahlen 89, 71, 92, 100 und 86 hat, welche Punktzahl benötigt sie dann im sechsten Test, um einen Durchschnitt von 93 zu erreichen?	<pre> :- use_module(library(clpqr)). num_tests(mr_bert, 6),lowest_score_removal(mr_bert, 1),target_average(mr_bert, 93). scores(brinley, [89, 71, 92, 100, 86]). solve(Test_score) :-num_tests(mr_bert, Num_tests),lowest_score_removal(mr_bert, Lowest_score_removal),target_average(mr_bert, Target_average),scores(brinley, Scores),Length is Num_tests - Lowest_score_removal,(Total_score = sum(Scores)), % Das eingebaute Prädikat wird missbraucht.(Average_score = Total_score / Length), {Test_score = (Target_average * Length) - Total_score}. </pre>

A.5 Trainingsdetails und Rechenbudget Während des Feinabstimmungsprozesses haben wir die Anzahl der Epochen auf 6, die Batch-Größe auf 128 und die Lernrate auf 3×10^{-4} festgelegt. Für einen einzelnen Trainingslauf verwendeten wir 2 NVIDIA RTX 4090 GPUs, um Llama2 und CodeLlama feinabzustimmen, und 2 NVIDIA RTX 8000 GPUs, um Mistral feinabzustimmen. Wir haben Distributed Data Parallelism verwendet, um das Training zu beschleunigen. Das Training mit den ursprünglichen CoT-Daten in GSM8K oder den nicht-permutierten Prolog-Code-Daten dauerte etwa 2 Stunden auf 2 NVIDIA RTX 4090 GPUs und etwa 10 Stunden auf 2 NVIDIA RTX 8000 GPUs. Als wir permutierte Stichproben hinzufügten, wuchs die Trainingszeit proportional zur Größe des Datensatzes, da wir die Anzahl der Epochen und die Batch-Größe kontrollierten. Während der Inferenz auf dem Testdatensatz verwendeten wir eine Batch-Größe von 2

auf einer RTX 4090 GPU, die etwa 6 Stunden für eine vollständige Inferenzrunde benötigte, und einer Batch-Größe von 3 auf einer RTX 8000 GPU, die etwa 7 Stunden für eine vollständige Inferenzrunde benötigte.